

Formelblad: OOP i Python

Disa Degerholm

Innehåll

1	Object-Oriented Programming (OOP)	2
1.1	Grundbegrepp	2
1.2	Syntax för Klasser och Objekt	2
2	Inkapsling och Arv	4
2.1	Privata Attribut och Metoder	4
2.2	Arv (Inheritance)	4
2.3	Klassdiagram & Liskovs Substitutionsprincip	5
2.4	Informationsfunktioner & MRO	5
2.5	Exempel: Geometriska Figurer (Arvs-kedja)	5
3	Avancerad OOP: Operatorer & Abstraktion	7
3.1	Implementera Jämförelser (Operator Overloading)	7
3.2	Exempel: Monster Go (Abstraktion & Arv)	7
3.3	Subklasser & Metodöverlagring (Overriding)	8
3.4	Aggregering & Gränssnitt (HackerDojo)	8
3.5	Olika typer av arv	9
3.5.1	Diamantproblem	9
3.6	Algebraiska uttryck & Rekursiva träd	10
3.6.1	Varför använda träd istället för strängar?	10
3.6.2	Implementering med operatorprioritet (Precedence)	10
3.7	Generella Binära Träd	11
4	Defensiv Programmering m.m	13
4.1	Defensiv programmering & Testning	13
4.1.1	Assert (Antaganden i koden)	13
4.1.2	Automatiserad testning	13

Kapitel 1

Object-Oriented Programming (OOP)

1.1 Grundbegrepp

Viktiga Begrepp

- **OOP:** `print(str)` kan ta in variabler med olika syntax. Funktion med specifika datatyper den tar in, t.ex. `write(filename)`.
- **Klass:** Regler (t.ex. `print`, `write`) – regler med tom data. `def` skapar objekt (referenser/kopia).
- **Objekt:** Kopia av klass / exempel på klass – regler med data (kopia + data).
- **Instans:** Ett specifikt objekt som man refererar till (t.ex. `mom`, `siblings`, `sibling`).
- **Attribut:** Variabler i objektet/klassen.
- **Metod:** Funktion som hör till klassen.

Intuition

En klass är regler utan data.
Ett objekt är en kopia av klassen fylld med data.
En instans är det specifika objektet du syftar på.

1.2 Syntax för Klasser och Objekt

```
1 # Syntax for klasser
2 class Bilar: # Bor namnges med stor bokstav i borjan, ska vara ett substantiv (t.ex. Bilar)
3
4     competition = "Formula 1" # Klassattribut, delad av alla objekt i klassen
5
6     def __init__(self, age, arg2): # Instansattribut, unik for varje objekt genom att data
7         # lags till
8         self.age = age # Har skapas attribut/variabler (t.ex. alder, vikt)
9         self.weight = arg2
10
11     def varde(self): # Funktion/metod som beraknar bilens varde
12         pengar = self.age * 1000 + self.weight * 500 # Obs: aven enkla funktioner som len
13         # () behovs laggas till sa de funkar pa din klass
14         return pengar
15
16     # Lagg till enkla funktioner vi vill ha
17     def __str__(self): # __str__() ar en speciell metod som definierar hur objektet ska
18         # representeras som en strang
19         return f"Bil med alder: {self.age}, vikt: {self.weight}, tavling: {Bilar.
20             competition}"
21
22     def __len__(self):
23         pass
24
25     def hoarse_happines(self):
26         return self.age      2
```

```

24 # Syntax for att skapa ett objekt (instans) av klassen
25 objekt = Bilar(2, 800) # Objekt kan nu manipulera hela datan från Bilar med hjälp av enkel
    punktnotation
26 objekt.attribut3 = 10 # Har kan man lagga till fler attribut, obs kan skrivas över
27 objekt.varde() # Har kan man anropa metoder som redan finns i klassen
28 objekt2 = Bilar(5, 1000) # Har skapas ett nytt objekt med annan data som en kopia
29 print(objekt2.age) # För att använda funktioner som redan finns på objektet, kalla på dess
    info/attribut
30
31 VolvoW80 = Bilar(2, 800)
32 volvo_pengar = VolvoW80.varde()
33 Hoarses = Bilar(7, 900)

```

Kapitel 2

Inkapsling och Arv

2.1 Privata Attribut och Metoder

```
1 class Bil:
2     def __init__(self, alder, vikt):
3         self._alder = alder # Privata attribut (bor inte andras utanfor klassen)
4         self._vikt = vikt   # Privata attribut (bor inte andras utanfor klassen)
5
6     def berakna_varde(self):
7         return self._alder * 1000 + self._vikt * 500 # Publik metod som anvander privata
8             attribut
9
10 min_bil = Bil(5, 1500) # Man kan manipulera med berakna_varde() men inte andra _alder
11                        eller _vikt direkt om man inte skrivs...
12 min_bil._alder = 10   # ...sa har, men det ar inte rekommenderat eftersom det bryter mot
13                        principen om inkapsling.
```

2.2 Arv (Inheritance)

- **Syfte:** Återanvända, lägga till eller ändra funktionalitet i befintliga klasser.
- **Termer:** Basklass / Superklass / Parent class (klassen man ärver från). Subklass / Child class (klassen som ärver).
- **Automatisk ärvning:** Metoder och klassattribut ärvs automatiskt. Instansattribut ärvs **inte** automatiskt om `__init__` definieras om i subklassen.
- **super():** Används i subklassens konstruktor för att anropa basklassens konstruktor och ärva dess instansattribut.

```
1 class A:
2     class_integer = 10 # attribut som arvs av alla subklasser
3     def __init__(self):
4         self.vals = [5] # attribut som inte arvs av subklasser om __init__ definieras om
5     def power_sum(self):
6         return sum([i ** 2 for i in self.vals]) + A.class_integer # self.vals=[5] -> 5 ** 2 +
7             10 = 35
8
9 class B(A): # B arver fran A
10     def __init__(self, x, y): # (self, x, y) kommer fran input och inte forbestams
11         self.vals = [x, y] # Definierar om instansattribut utan super()
12     def print_result(self):
13         print(self.power_sum()) # Anvander metod fran A
14
15 class C(A):
16     def __init__(self, y):
17         super().__init__() # Anropar A:s __init__ sa self.vals skapas!
18         self.vals.append(y)
19
20 a = A() # a.power_sum() -> 35
21 b = B(10, 20) # b.power_sum() -> 510
22 c = C(10) # c.vals -> [5, 10], c.power_sum() -> 135
```

2.3 Klassdiagram & Liskovs Substitutionsprincip

- **Klassdiagram (UML):** Visualiserar hur klasser hänger ihop oavsett tidpunkt (visar relationer mellan klasser som "is-a", medan om en klass innehåller ett objekt från en annan klass med **komposition**). Ett **objektdiagram** visar tillståndet i ett program vid en viss tidpunkt.
- **Liskovs substitutionsprincip:** Allt som gäller för en basclass ska också gälla för dess subklasser.

2.4 Informationsfunktioner & MRO

- **isinstance(obj, Class):** Returnerar True om obj är en instans av Class eller dess superklasser.
- **issubclass(Sub, Super):** Returnerar True om klassen Sub ärver från Super.
- **MRO (Method Resolution Order):** Den ordning Python söker efter metoder i arvskedjan. Kan visas med Klass.mro().
- **Object:** Basklassen för alla typer och klasser i Python ("Everything is an object in Python").

```
1 isinstance(a, A)          # True
2 isinstance(c, A)          # True (c ärver från A)
3 issubclass(B, A)          # True
4 issubclass(str, object) # True
5
6 print(B.mro()) # [__main__.B, __main__.A, object]
```

2.5 Exempel: Geometriska Figurer (Arvs-kedja)

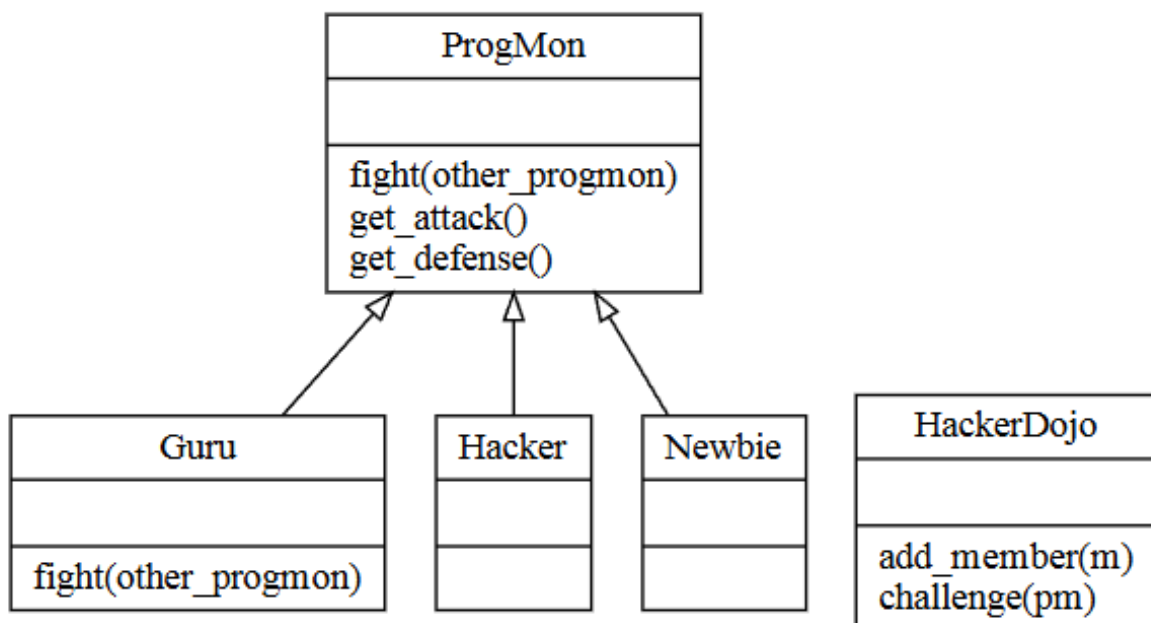
Exemplet visar hur en basclass Polygon utökas och specialiseras av Triangle, Rectangle och Square med hjälp av metodöverlagring och super().

```
1 class Polygon:
2     __name__ = "Polygon" # Hanterar endast symmetriska former
3     def __init__(self, n_sides, sides=None):
4         self.n_sides = n_sides # Antal sidor (int)
5         self.sides = sides if sides else [1 for _ in range(n_sides)] # Lista med
6             sidlangder (float)
7         self.check_sides() # kallar på check_sides() för att validera sidlangderna
8
9     def check_sides(self):
10         if not self.sides or not all(map(lambda x: x > 0, self.sides)):
11             raise ValueError("Invalid side lengths")
12         if self.n_sides != len(self.sides):
13             raise ValueError("Wrong number of sides")
14
15     def set_sides_from_input(self, n_unique_sides=None): # målet är att kunna mata in t.ex
16         . 2 unika sidor för en rektangel (dvs 2 sidor är lika långa) och få ut sides som [
17         a, b, a, b] (dvs 4 sidor totalt)
18         if not n_unique_sides: n_unique_sides = self.n_sides # Om inget valts: anta att
19             alla sidor är unika -> inlasning av lika många unika langder som totalt antal
20             sidor
21         if self.n_sides % n_unique_sides != 0: # Om totala antalet sidor inte är en
22             multipel av unika sidor: fel
23             raise ValueError("Total sides must be multiple of unique sides")
24         unique = [float(input(f"Enter side {i+1}: ")) for i in range(n_unique_sides)] #
25             las in en sida i taget för de n_unique_sides
26         self.sides = unique * (self.n_sides // n_unique_sides) # returnera antal sidor i
27             en lista med de unika langderna upprepade så att totala antalet sidor blir
28             n_sides, e.x. [a, b] * 2 = [a, b, a, b] för en rektangel med 4 sidor
29         self.check_sides() # se sidor är validerade
30
31 class Triangle(Polygon):
32     __name__ = "Triangle"
33     def __init__(self, sides=None):
34         super().__init__(3, sides)
35
36     def check_sides(self):
37         a, b, c = sorted(self.sides)
```

```

29         if not (c < a + b):
30             raise ValueError(f"Shortest sides must be > longest in {self.__name__}") # Har
31             betyder self.__name__ Triangle
32         super().check_sides() # Har fortsatter checken mha gamla testerna
33
34     def get_area(self):
35         a, b, c = sorted(self.sides)
36         s = (a + b + c) / 2
37         return (s * (s - a) * (s - b) * (s - c)) ** 0.5
38
39 class Rectangle(Polygon):
40     __name__ = "Rectangle"
41     def __init__(self, sides=None):
42         super().__init__(4, sides)
43
44     def check_sides(self):
45         a, b, c, d = self.sides
46         if not (a == c and b == d):
47             raise ValueError(f"Opposite sides must be equal in {self.__name__}")
48         super().check_sides()
49
50     def get_area(self):
51         return self.sides[0] * self.sides[1]
52
53     def get_diagonal(self):
54         return (self.sides[0] ** 2 + self.sides[1] ** 2) ** 0.5
55
56 class Square(Rectangle):
57     __name__ = "Square"
58     def __init__(self, sides=None):
59         super().__init__(sides)
60
61     def check_sides(self):
62         a, b, c, d = self.sides
63         if not (a == b == c == d):
64             raise ValueError(f"All sides must be equal in {self.__name__}")
65         super().check_sides()
66
67     def get_corner_coordinates(self):
68         s = self.sides[0] / 2
69         return ((-s, -s), (-s, s), (s, s), (s, -s))

```



Kapitel 3

Avancerad OOP: Operatorer & Abstraktion

3.1 Implementera Jämförelser (Operator Overloading)

- Metoder som `__lt__` (<), `__gt__` (>) och `__eq__` (==) gör det möjligt att jämföra objekt.
- Om metoderna definieras i en basclass ärvs de automatiskt av alla subklasser.

```
1 # Implementeras t.ex. i Triangle och Rectangle (Square arver automatiskt):
2 def __lt__(self, other):
3     return self.get_area() < other.get_area()
4
5 def __gt__(self, other):
6     return self.get_area() > other.get_area()
7
8 def __eq__(self, other):
9     return self.get_area() == other.get_area()
```

3.2 Exempel: Monster Go (Abstraktion & Arv)

- Syfte: Illustrera värdet av arv och abstraktion (dölja implementationsdetaljer via ett fast gränssnitt).
- Privata attribut/metoder: Inleds med `_` (t.ex. `_attack`) eller magiska metoder (`__str__`).

```
1 class ProgMon:
2     def __init__(self):
3         self._attack = 0.0
4         self._defense = 0.0
5         self._caffeinated = False
6         self._unit_testing = False
7
8     def __str__(self):
9         return "<ProgMon object>"
10
11     def get_attack(self):
12         return 2 * self._attack if self._caffeinated else self._attack
13
14     def get_defense(self):
15         return 2 * self._defense if self._unit_testing else self._defense
16
17     def fight(self, other_progmon):
18         if self.get_attack() > other_progmon.get_defense():
19             return (True, False) # self vinner, return som tulp
20         elif other_progmon.get_attack() > self.get_defense():
21             return (False, True) # other vinner
22         else:
23             return (False, False) # Avgjort
24
25     , , ,
```



```

26 Skapa monster: p1 = ProgMon() -> Andra status: p1._caffeinated = True -> p1.fight(p2) kors
27 -> p1.get_attack() (dubblerad) jamfors mot p2.get_defense() -> returnerar resultat som
28 tuppel: (True, False)
29 '''

```

3.3 Subklasser & Metodöverlagring (Overriding)

- **Specialisering:** Subklasser sätter sina egna värden på skyddade attribut och kan skriva om metoder.
- **Metodöverlagring:** Om en subklass definierar om en metod (t.ex. `fight` i `Guru`) ändras beteendet utan att anropande kod behöver ändras.

```

1 class Hacker(ProgMon):
2     def __init__(self):
3         super().__init__()
4         self._attack = 0.5
5         self._defense = 0.25
6     def __str__(self):
7         return "<Hacker A=" + str(self._attack) + ">" # print(Hacker()) -> <Hacker A=0.5>
8
9 class Newbie(ProgMon):
10    def __init__(self):
11        super().__init__()
12        self._attack = 0.15
13        self._defense = 0.1
14    def __str__(self):
15        return "<Newbie>"
16
17 class Guru(ProgMon):
18    def __init__(self):
19        super().__init__()
20        self._attack = 1.0
21        self._defense = 1.0
22
23    # Overlagrar fight() med egna regler ("Don't win, don't lose")
24    def fight(self, other_progmon):
25        return (False, False)

```

3.4 Aggregering & Gränssnitt (HackerDojo)

- **HackerDojo:** Innehåller en samling av `ProgMon`-objekt (aggregering).
- **Framtidssäkring via abstraktion:** `HackerDojo` behöver bara känna till gränssnittet (`.fight()`) – inte om monstret är en `Hacker`, `Newbie` eller `Guru`.

```

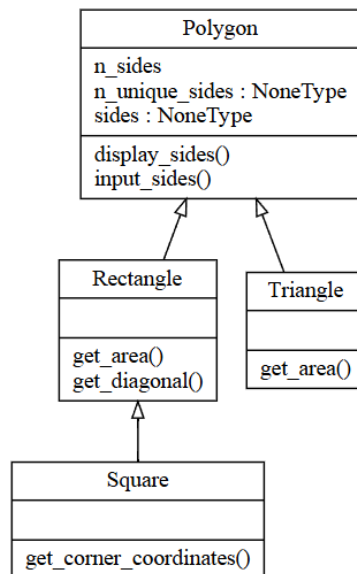
1 class HackerDojo:
2     def __init__(self):
3         self._members = []
4
5     def add_member(self, m):
6         self._members.append(m)
7
8     def challenge(self, pm):
9         for monster in self._members:
10            win, loose = pm.fight(monster)
11            if loose: # dvs (nagot, True)
12                return False # Forlorade mot minst en medlem
13            return True # Klarade alla matcher utan forlust
14
15 # Användning:
16 dojo = HackerDojo()
17 dojo.add_member(Hacker())
18 dojo.add_member(Newbie())
19
20 g = Guru()
21 n = Newbie()

```

```

22 print(dojوchallenge(g)) # True (Guru forlorar aldrig pa grund av sin overlgrade fight-
    metod)
23 print(dojوchallenge(n)) # False

```



3.5 Olika typer av arv

I förra kapitlet såg vi olika exempel på arv i Python. Olika former av arv har olika namn inom objektorientering:

1. **Enkelt arv** (eng. *single inheritance*): klass B ärver endast från klass A.
2. **Mångnivåarv** (eng. *multilevel inheritance*): klass B ärver från A och klass C ärver sen från klass B.
3. **Hierarkiskt arv** (eng. *hierarchical inheritance*): klass A är superklass till B, C, D...
4. **Multipelt arv** (eng. *multiple inheritance*): klass C ärver från *både* A och B (men de ärver inte från varandra).
5. **Hybridarv** (eng. *hybrid inheritance*): en mix av två eller fler typer av arv.

Vi såg även att man kan illustrera olika typer av arv med hjälp av klassdiagram. Förra kapitlet innehöll exempel på enkelt, mångnivå-, hierarkiskt och hybrid-arv, så vi ska nu titta på multipelt arv.

Det är inte alla programmeringsspråk som tillåter multipelt arv, men det gör Python. Betrakta följande exempel:

```

1 class A:
2     pass # some code
3
4 class B:
5     pass # some more code
6
7 class C(A, B):
8     pass # even more code

```

Här har C alltså två superklasser, A och B, vilka inte ärver från varandra. Detta kan verka oskyldigt och man kan undra varför inte alla språk stödjer det. Men rent generellt är det lätt hänt att multipelt arv ger kaos på grund av så kallade *diamantproblem*.

3.5.1 Diamantproblem

Betrakta följande kod:

```

1 class A:
2     def f(self):
3         print("f of A called")
4
5 class B(A):
6     def f(self):
7         print("f of B called")

```

```

8
9 class C(A):
10     def f(self):
11         print("f of C called")
12
13 class D(B, C):
14     pass
15
16 d = D()
17 d.f() # Ger f och B called och om C, B byter plats ges f och C called

```

Så utdata har nu ändrats då man ärver i en annan ordning! Denna typ av beroenden vill man helst inte behöva tänka på när man programmerar och därför tycker man att det är ett problem.

3.6 Algebraiska uttryck & Rekursiva träd

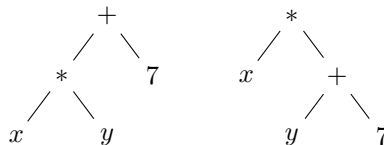
Genom att kombinera klasser och rekursion kan vi effektivt representera och manipulera hierarkisk data, som algebraiska uttryck och binära träd.

3.6.1 Varför använda träd istället för strängar?

Att representera algebraiska uttryck (t.ex. $x \cdot y + 7$) som vanliga strängar skapar problem vid utvärdering och variabelbyten. Strängar saknar strukturell information om t.ex. operatorprioritet och parenteser.

Ett binärt träd löser detta genom att representera uttrycket i två dimensioner:

- Noder: Operatorer (+, *) som har vänster och höger underträd.
- Löv: Värden och variabler (7, x, y) utan underträd.



3.6.2 Implementering med operatorprioritet (Precedence)

För att skriva ut uttrycken korrekt med parenteser tilldelas varje operator ett prioritetsvärde (`prec`). Om ett underträd har lägre prioritet än sin förälder omsluts det med parenteser.

```

1 class Expr: # star for Expression
2     prec = 1000 # prec star for precedence, har hog
3
4 class Constant(Expr):
5     def __init__(self, value):
6         self._value = value
7     def __str__(self):
8         return str(self._value)
9
10 class Var(Expr):
11     def __init__(self, name):
12         self._name = name
13     def __str__(self):
14         return self._name
15
16 class BinOp(Expr):
17     def __init__(self, left, right):
18         self._left = left
19         self._right = right
20
21 def parens(p1, p2, s): # parens star for parenteser
22     return f"({s})" if p1 < p2 else s # p1 star for prioritet for foralder, p2 for
23     # prioritet for barn
24     # Ex man vill satta parantes pa plus
25
26 class Plus(BinOp):
27     prec = 1

```

```

27     def __str__(self):
28         s1 = parens(self._left.prec, self.prec, str(self._left))
29         s2 = parens(self._right.prec, self.prec, str(self._right))
30         return f"{s1} + {s2}"
31 # bygger en teststrang av två uttryck genom att kontrollera om vanster- och hogerledet
32 # behöver omges av parenteser utifrån deras prioritetssvården (prec).
33
34 class Times(BinOp):
35     prec = 2
36     def __str__(self):
37         s1 = parens(self._left.prec, self.prec, str(self._left))
38         s2 = parens(self._right.prec, self.prec, str(self._right))
39         return f"{s1} * {s2}"
40
41 # Exempel på anrop och automatisk parenteshantering (läses från höger till vänster):
42 print(Plus(Times(Var("x"), Var("y")), Constant(7))) # Out: x * y + 7
43 print(Times(Var("x"), Plus(Var("y"), Constant(7)))) # Out: x * (y + 7)

```

3.7 Generella Binära Träd

Mönstret ovan kan generaliseras till godtyckliga binära träd med rekursiva metoder för databearbetning.

- `__str__()`: Formaterar trädet rekursivt som en sträng.
- `member(x)`: Söker rekursivt i trädet efter värdet `x`.
- `map_tree(f)`: Högre ordningens funktion som applicerar funktionen `f` på varje nod/löv.
- `linearize()`: Gör om trädstrukturen till en platt lista (in-order).

```

1 class BinTree:
2     pass
3
4 class Branch(BinTree):
5     def __init__(self, node, left, right):
6         self._node = node
7         self._left = left
8         self._right = right
9
10    def __str__(self):
11        return f"({self._node},{self._left},{self._right})"
12
13    def member(self, x):
14        return self._node == x or self._left.member(x) or self._right.member(x)
15
16    def map_tree(self, f):
17        return Branch(f(self._node), self._left.map_tree(f), self._right.map_tree(f))
18
19    def linearize(self):
20        return self._left.linearize() + [self._node] + self._right.linearize()
21
22 class Leaf(BinTree):
23     def __init__(self, val):
24         self._val = val
25
26    def __str__(self):
27        return f"({self._val})"
28
29    def member(self, x):
30        return self._val == x
31
32    def map_tree(self, f):
33        return Leaf(f(self._val))
34
35    def linearize(self):
36        return [self._val]
37
38 # Test och korning
39 t = Branch(-32, Leaf(2), Branch(1, Branch(23, Leaf(4), Leaf(-2)), Leaf(12)))

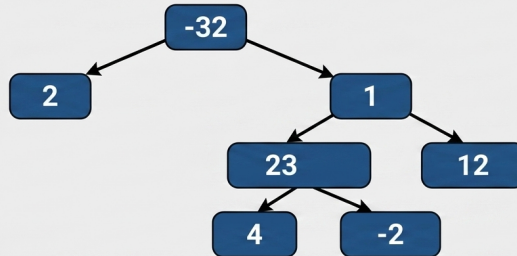
```

```

40
41 print(t)                                # (-32, (2), (1, (23, (4), (-2)), (12)))
42 print(t.member(12))                    # True
43 print(t.map_tree(lambda x: x 2))      # Kvadrat pa alla element
44 print(t.linearize())                   # [2, -32, 4, 23, -2, 1, 12]

```

Tree Structure from `#(-32, (2), (1, (23, (4), (-2)), (12)))`



Source: `#(-32, (2), (1, (23, (4), (-2)), (12)))`

Kapitel 4

Defensiv Programering

4.1 Defensiv programmering & Testning

Defensiv programmering handlar om att koda sina antaganden för att upptäcka fel tidigt, nära felkällan, och undvika långvarig felsökning.

4.1.1 Assert (Antaganden i koden)

- **Syfte:** Verifiera att internt tillstånd är korrekt under utveckling/felsökning.
- **Beteende:** Om villkoret är falskt kastar Python ett `AssertionError`.
- **Syntax:** `assert condition, [Optional error message]`
- **Optimeringsflagga:** Körs programmet med `python -O script.py` inaktiveras alla `assert`.

```
1 def divide(x, y):
2     # Ensure denominator is non-zero
3     assert y != 0, "Second argument cannot be 0 when dividing"
4     return x / y
```

Klassificering av villkor

- **Förvillkor (Preconditions):** Krav på indata som måste gälla när en funktion anropas.
- **Eftervillkor (Postconditions):** Krav på utdata/resultat när en funktion returnerar.
- **Invarianter (Loop/Class Invariants):** Tillstånd som alltid måste hålla under körning eller mellan metदानrop.

```
1 class MyDB: # DB stands for database
2     def __init__(self):
3         self._id2name_map = {}
4         self._name2id_map = {}
5
6     def add(self, id, name):
7         # Preconditions: Type checks
8         assert isinstance(id, int), "ID must be integer" # isinstance() returns True or
9             False
10        assert isinstance(name, str), "Name must be string"
11        self._name2id_map[name] = id
12        self._id2name_map[id] = name
13
14    def by_name(self, name):
15        id = self._name2id_map[name]
16        # Invariant check: Bi-directional map must match
17        assert self._id2name_map[id] == name # checks if the id maps back to the same name
18        return id
```

4.1.2 Automatiserad testning

För att säkerställa att ny funktionalitet inte förstör befintlig kod (*regression testing*) används automatiserade tester.

Enhetstester med unittest

- **Struktur:** Tester grupperas som metoder i en klass som ärver från `unittest.TestCase`.
- **Assertion-metoder:** Använder specifika tester som `self.assertEqual(a, b)`, `self.assertTrue(x)`, etc.
- **Körning:** Startas med `unittest.main()` och om allt går rätt står `Ran X tests in Y s` och `ÖK` i terminalen.

```
1 import unittest
2
3 class TestPolynomialOperations(unittest.TestCase):
4     def test_addition(self):
5         # Verify that adding polynomials is symmetric
6         p = [2, 0, 1]
7         q = [-2, 1, 0, 1]
8         self.assertEqual(add_poly(p, q), add_poly(q, p), "Function should be symmetric")
9
10    def test_evaluation(self):
11        # Verify polynomial evaluation at x=0
12        p = [2, 0, 1]
13        self.assertEqual(eval_poly(p, 0), 2, "Should evaluate to constant term")
14
15 if __name__ == '__main__':
16     unittest.main() # Run all defined test methods
```

Slumptestning (Property-Based Testing)

Istället för att skriva specifika testfall för hand definieras generella egenskaper (specifikationer) som koden måste uppfylla. Bibliotek som `hypothesis` genererar automatiskt slumpmässig indata för att hitta gränsfall där koden fallerar.